DATA SATURDAYS #64 PARMA 2024

Don't Repeat Yourself!

how custom Python modules in Microsoft Fabric gives you back hours every day

MICROSOFT FABRIC


Bas Land
Don’t Repeat Yourself

Sponsor & Org

iSolutions

engage LABS

DATA SKILLS
UNDERSTANDING THE WORLD

LIME ware

daniela malvisi
COMMUNICATION

Lucient ITALIA

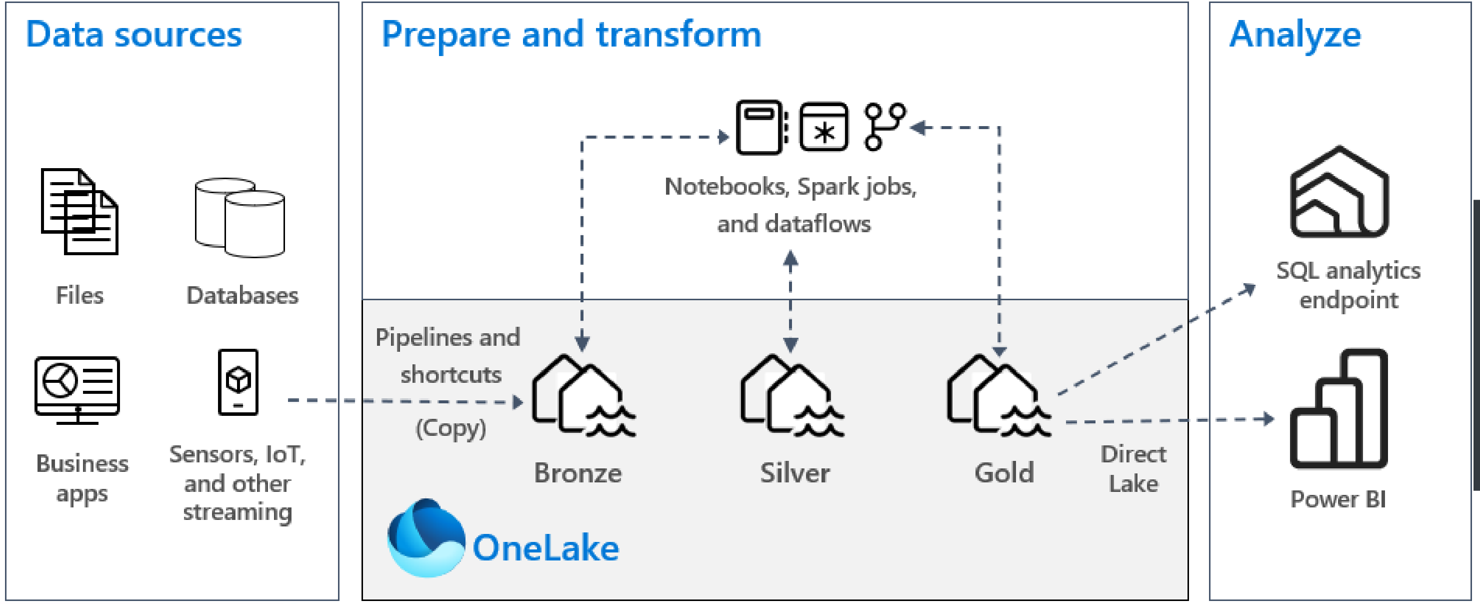
Bi Factory
DATA KNOWLEDGE ADVISOR

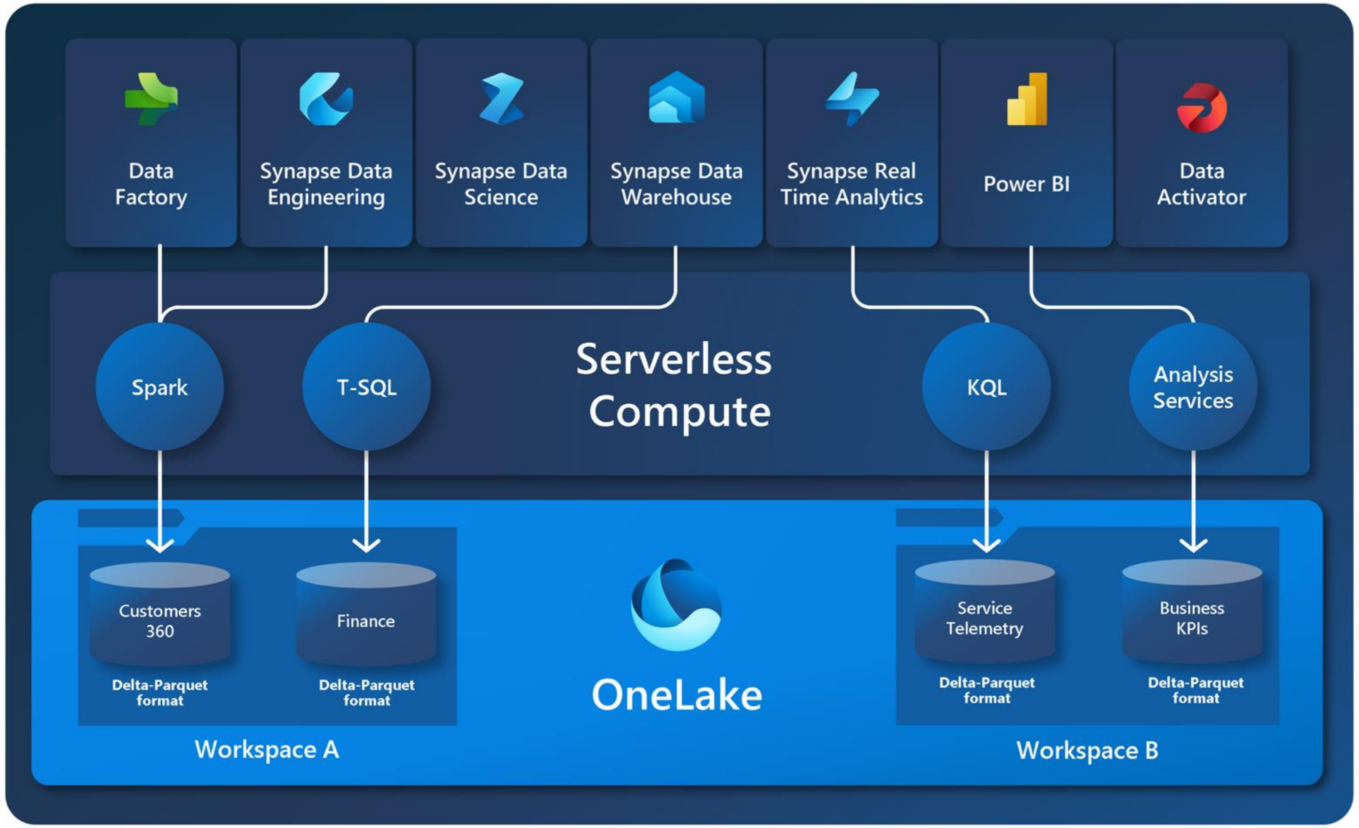
Today

- Core concepts of Fabric compute
- The Problem: Repetitive Coding
- The Solution: Reusable Python Modules
- Live Demo
- Q&A

Core concepts
Fabric Compute

Fabric compute





Python in Fabric

- Python and Spark work together quite nicely!

- Benefits of using Python:
  - Extensive libraries for common tasks in data engineering, data science, statistics
  - Community support (probably the most popular programming language)
  - Flexibility in data engineering tasks

Python examples (1)

```python
one = 1
two = 2
total = one + two
message = f"Hello world. The total is {total}."

print(message)
```

```python
one = 1
two = 2
total = one + two
message = f"Hello world. The total is {total}."

print(message)
```

[129] ✓ 1 sec - Command executed in 302 ms by Olav van Rabenswaaij | Kimura on 3:08:32 PM, 11/18/24

```
Hello world. The total is 3.
```

Python examples (2)

```python
columns = ['ID', 'Name', 'Age']
data = [
    (1, 'Alice', 28),
    (2, 'Bob', 35),
    (3, 'Charlie', 23)
]

df = spark.createDataFrame(data, columns)

df.show()
```

[2] ✓ 2 sec - Command executed in 1 sec 595 ms by admin-kimura on 4:29:33 PM, 11/18/24

Spark jobs (3 of 3 succeeded) Resources Log

```
+---+-------+---+
| ID|   Name|Age|
+---+-------+---+
|  1|  Alice| 28|
|  2|    Bob| 35|
|  3|Charlie| 23|
+---+-------+---+
```

Python examples (3)

```python
import re
from datetime import datetime

#Table to load
tables = [
    {'endpoint': 'invoices', 'table_key': "['id']", 'json_root_array': 'results'},
    #{'endpoint': 'tasks', 'table_key': "['id']", 'json_root_array': 'results'},
    {'endpoint': 'registrations', 'table_key': "['id']", 'json_root_array': 'results'},
    {'endpoint': 'registrations/pricelistitems', 'table_key': "['id']", 'json_root_array': ''},
    {'endpoint': 'relations', 'table_key': "['uniqueIdentifier']", 'json_root_array': 'results'},
    {'endpoint': 'users', 'table_key': "['id']", 'json_root_array': 'results'},
    #{'endpoint': 'users/teams', 'table_key': "['id']", 'json_root_array': ''},
    {'endpoint': 'companies', 'table_key': "['id']", 'json_root_array': ''}
]

#Work
activities = []

#Load data to Silver layer delta tables
for table in tables:
    endpoint = table['endpoint']
    table_key = table['table_key']

    args = {
        'target_table': endpoint,
        'source_system': "AdminPulse",
        'file_type': "json",
        'table_key_list': table_key,
        'timestamp': timestamp
    }

    activity = {
        'name': endpoint,
        'path': 'AP_Silver_Worker_FullLoad',
        'args': args,
        'timeoutPerCellInSeconds': 9000,
        'retry': 1,
        'retryIntervalInSeconds': 10,
        'dependencies': []
    }
    activities.append(activity)
```

The problem:
Repetitive coding

Repetitive code

- Common challenges:
  - Duplicated code across processes
  - Manual changes leading to errors
  - Difficulty in scaling
- Impact on productivity and developer’s happiness

Real-world scenario

- Extending a dimension table in the data warehouse
- Extracting a couple of new tables
  - Making extract scripts
  - Making scripts for the Silver/persistent layer
- Adding this data to the dimension
  - Adding the join
  - Putting new fields in the mapping
  - Also in the insert/update merge, in the unknown member, etc
- Lots of work!

Cost of repetition

- Time spent on writing and debugging repetitive code
- Increased risk of inconsistencies and bugs
- Difficulty maintaining and extending the codebase

Don’t repeat yourself (DRY)

- A principal aimed at reducing repetition
- Important in software development – let’s learn from this as data engineers
- Enhances code maintainability
- Facilitates scalability

Don’t repeat yourself (DRY)

- Every piece of knowledge must have a **single**, **unambiguous**, **authoritative** representation within a system

- Single: everything should be configured in just one location/script
  - Examples: connection strings, file paths, etc
- Unambiguous: everything should be clearly named and documented
  - Examples: Python docstrings
- Authoritative: single source of truth for data models, objects, etc
  - Examples: Process/notebook to generate Silver layer tables

The solution:
Reusable Python modules

Custom Python modules

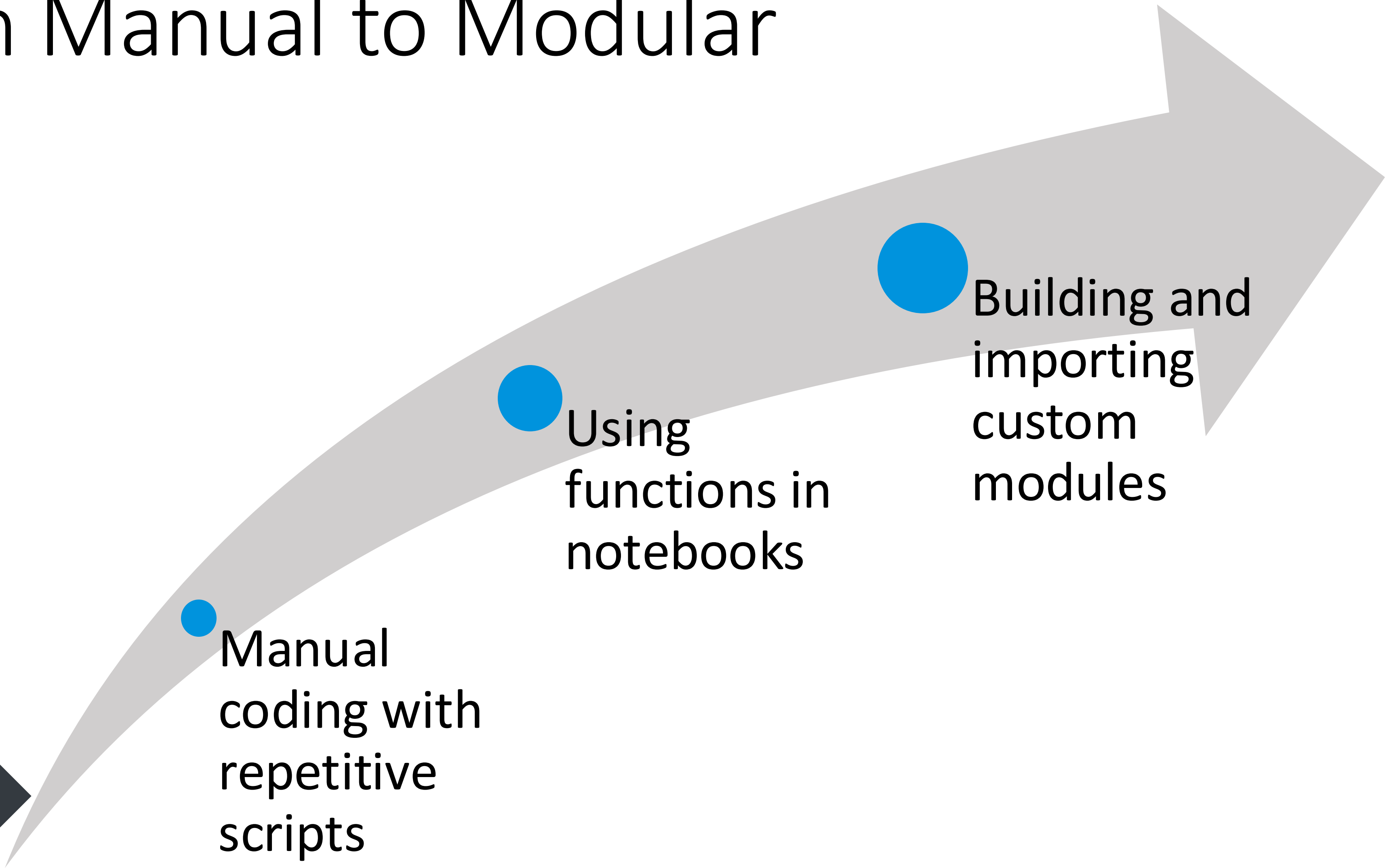
- Finding and using common functions and procedures
- Sharing modules across projects

- Benefits:
  - Saves time
  - Reduces errors
  - Simplifies updates

Why use custom modules?

- Reusability: write once, use everywhere
- Consistency: standardises processes across projects
- Efficiency: accelerates development cycles (less time spent on ‘reinventing the wheel’)
- Collaboration: easier for teams to work together

From Manual to Modular



Stage 1 – Manual Coding

```
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Load data from CSV files for three different departments
df_sales = spark.read.csv("/lakehouse/data/sales_data.csv", header=True, inferSchema=True)
df_marketing = spark.read.csv("/lakehouse/data/marketing_data.csv", header=True, inferSchema=True)
df_hr = spark.read.csv("/lakehouse/data/hr_data.csv", header=True, inferSchema=True)

# Transform sales data
df_sales = df_sales.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_sales = df_sales.withColumn("revenue", col("revenue").cast("double"))
df_sales = df_sales.filter(col("revenue") > 0)
df_sales = df_sales.withColumnRenamed("revenue", "amount")
df_sales = df_sales.select("date", "amount", "product_id", "customer_id")

# Transform marketing data
df_marketing = df_marketing.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_marketing = df_marketing.withColumn("cost", col("cost").cast("double"))
df_marketing = df_marketing.filter(col("cost") > 0)
df_marketing = df_marketing.withColumnRenamed("cost", "amount")
df_marketing = df_marketing.select("date", "amount", "campaign_id")

# Transform HR data
df_hr = df_hr.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_hr = df_hr.withColumn("salary", col("salary").cast("double"))
df_hr = df_hr.filter(col("salary") > 0)
df_hr = df_hr.withColumnRenamed("salary", "amount")
df_hr = df_hr.select("date", "amount", "employee_id")
```

Stage 1 – Manual Coding

```python
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Load data from CSV files for three different departments
df_sales = spark.read.csv("/lakehouse/data/sales_data.csv", header=True, inferSchema=True)
df_marketing = spark.read.csv("/lakehouse/data/marketing_data.csv", header=True, inferSchema=True)
df_hr = spark.read.csv("/lakehouse/data/hr_data.csv", header=True, inferSchema=True)

# Transform sales data
df_sales = df_sales.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_sales = df_sales.withColumn("revenue", col("revenue").cast("double"))
df_sales = df_sales.filter(col("revenue") > 0)
df_sales = df_sales.withColumnRenamed("revenue", "amount")
df_sales = df_sales.select("date", "amount", "product_id", "customer_id")

# Transform marketing data
df_marketing = df_marketing.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_marketing = df_marketing.withColumn("cost", col("cost").cast("double"))
df_marketing = df_marketing.filter(col("cost") > 0)
df_marketing = df_marketing.withColumnRenamed("cost", "amount")
df_marketing = df_marketing.select("date", "amount", "campaign_id")

# Transform HR data
df_hr = df_hr.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_hr = df_hr.withColumn("salary", col("salary").cast("double"))
df_hr = df_hr.filter(col("salary") > 0)
df_hr = df_hr.withColumnRenamed("salary", "amount")
df_hr = df_hr.select("date", "amount", "employee_id")
```

Stage 1 – Manual Coding

```python
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Load data from CSV files for three different departments
df_sales = spark.read.csv("/lakehouse/data/sales_data.csv", header=True, inferSchema=True)
df_marketing = spark.read.csv("/lakehouse/data/marketing_data.csv", header=True, inferSchema=True)
df_hr = spark.read.csv("/lakehouse/data/hr_data.csv", header=True, inferSchema=True)

# Transform sales data
df_sales = df_sales.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_sales = df_sales.withColumn("revenue", col("revenue").cast("double"))
df_sales = df_sales.filter(col("revenue") > 0)
df_sales = df_sales.withColumnRenamed("revenue", "amount")
df_sales = df_sales.select("date", "amount", "product_id", "customer_id")

# Transform marketing data
df_marketing = df_marketing.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_marketing = df_marketing.withColumn("cost", col("cost").cast("double"))
df_marketing = df_marketing.filter(col("cost") > 0)
df_marketing = df_marketing.withColumnRenamed("cost", "amount")
df_marketing = df_marketing.select("date", "amount", "campaign_id")

# Transform HR data
df_hr = df_hr.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
df_hr = df_hr.withColumn("salary", col("salary").cast("double"))
df_hr = df_hr.filter(col("salary") > 0)
df_hr = df_hr.withColumnRenamed("salary", "amount")
df_hr = df_hr.select("date", "amount", "employee_id")
```

Stage 2 – Using Functions

```python
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Define a function to read CSV files
def read_csv_file(file_path):
    return spark.read.csv(file_path, header=True, inferSchema=True)

# Define a function to transform DataFrames
def transform_dataframe(df, amount_column):
    df = df.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
    df = df.withColumn("amount", col(amount_column).cast("double"))
    df = df.filter(col("amount") > 0)
    df = df.select("date", "amount", *[col for col in df.columns if col not in ["date", amount_column]])
    return df

# List of datasets with their respective file paths and amount columns
datasets = [
    {"name": "sales", "file_path": "/lakehouse/data/sales_data.csv", "amount_column": "revenue"},
    {"name": "marketing", "file_path": "/lakehouse/data/marketing_data.csv", "amount_column": "cost"},
    {"name": "hr", "file_path": "/lakehouse/data/hr_data.csv", "amount_column": "salary"},
]

# Read and transform each dataset using the defined functions
transformed_dfs = []

for dataset in datasets:
    df = read_csv_file(dataset["file_path"])
    df = transform_dataframe(df, dataset["amount_column"])
    transformed_dfs.append(df)
```

Stage 2 – Using Functions

```python
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Define a function to read CSV files
def read_csv_file(file_path):
    return spark.read.csv(file_path, header=True, inferSchema=True)

# Define a function to transform DataFrames
def transform_dataframe(df, amount_column):
    df = df.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
    df = df.withColumn("amount", col(amount_column).cast("double"))
    df = df.filter(col("amount") > 0)
    df = df.select("date", "amount", *[col for col in df.columns if col not in ["date", amount_column]])
    return df

# List of datasets with their respective file paths and amount columns
datasets = [
    {"name": "sales", "file_path": "/lakehouse/data/sales_data.csv", "amount_column": "revenue"},
    {"name": "marketing", "file_path": "/lakehouse/data/marketing_data.csv", "amount_column": "cost"},
    {"name": "hr", "file_path": "/lakehouse/data/hr_data.csv", "amount_column": "salary"},
]

# Read and transform each dataset using the defined functions
transformed_dfs = []

for dataset in datasets:
    df = read_csv_file(dataset["file_path"])
    df = transform_dataframe(df, dataset["amount_column"])
    transformed_dfs.append(df)
```

Stage 2 – Using Functions

```python
# Import necessary PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# Define a function to read CSV files
def read_csv_file(file_path):
    return spark.read.csv(file_path, header=True, inferSchema=True)

# Define a function to transform DataFrames
def transform_dataframe(df, amount_column):
    df = df.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
    df = df.withColumn("amount", col(amount_column).cast("double"))
    df = df.filter(col("amount") > 0)
    df = df.select("date", "amount", *[col for col in df.columns if col not in ["date", amount_column]])
    return df

# List of datasets with their respective file paths and amount columns
datasets = [
    {"name": "sales", "file_path": "/lakehouse/data/sales_data.csv", "amount_column": "revenue"},
    {"name": "marketing", "file_path": "/lakehouse/data/marketing_data.csv", "amount_column": "cost"},
    {"name": "hr", "file_path": "/lakehouse/data/hr_data.csv", "amount_column": "salary"},
]

# Read and transform each dataset using the defined functions
transformed_dfs = []

for dataset in datasets:
    df = read_csv_file(dataset["file_path"])
    df = transform_dataframe(df, dataset["amount_column"])
    transformed_dfs.append(df)
```

Stage 3 – Using Custom Modules

```python
# Import necessary PySpark functions
from pyspark.sql.functions import col, to_date

def read_csv_file(spark, file_path):
    """
    Reads a CSV file into a Spark DataFrame.

    Parameters:
    - spark: The SparkSession object.
    - file_path (str): The path to the CSV file.

    Returns:
    - DataFrame: The loaded DataFrame.
    """
    return spark.read.csv(file_path, header=True, inferSchema=True)

def transform_dataframe(df, amount_column):
    """
    Transforms a DataFrame by processing date and amount columns.

    Parameters:
    - df (DataFrame): The DataFrame to transform.
    - amount_column (str): The name of the column representing the amount.

    Returns:
    - DataFrame: The transformed DataFrame.
    """
    df = df.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
    df = df.withColumn("amount", col(amount_column).cast("double"))
    df = df.filter(col("amount") > 0)
    df = df.select("date", "amount", *[col for col in df.columns if col not in ["date", amount_column]])
    return df
```

Stage 3 – Using Custom Modules

```python
# Import necessary PySpark functions and custom module
from pyspark.sql import SparkSession
from data_utils import read_csv_file, transform_dataframe

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# List of datasets with their respective file paths and amount columns
datasets = [
    {"name": "sales", "file_path": "/lakehouse/data/sales_data.csv", "amount_column": "revenue"},
    {"name": "marketing", "file_path": "/lakehouse/data/marketing_data.csv", "amount_column": "cost"},
    {"name": "hr", "file_path": "/lakehouse/data/hr_data.csv", "amount_column": "salary"},
]

# Read and transform each dataset using the imported functions
transformed_dfs = []

for dataset in datasets:
    df = read_csv_file(spark, dataset["file_path"])
    df = transform_dataframe(df, dataset["amount_column"])
    transformed_dfs.append(df)
```

Stage 3 – Using Custom Modules

```python
# Import necessary PySpark functions
from pyspark.sql.functions import col, to_date

def read_csv_file(spark, file_path):
    """
    Reads a CSV file into a Spark DataFrame.

    Parameters:
    - spark: The SparkSession object.
    - file_path (str): The path to the CSV file.

    Returns:
    - DataFrame: The loaded DataFrame.
    """
    return spark.read.csv(file_path, header=True, inferSchema=True)

def transform_dataframe(df, amount_column):
    """
    Transforms a DataFrame by processing date and amount columns.

    Parameters:
    - df (DataFrame): The DataFrame to transform.
    - amount_column (str): The name of the column representing the amount.

    Returns:
    - DataFrame: The transformed DataFrame.
    """
    df = df.withColumn("date", to_date(col("date"), "MM/dd/yyyy"))
    df = df.withColumn("amount", col(amount_column).cast("double"))
    df = df.filter(col("amount") > 0)
    df = df.select("date", "amount", *[col for col in df.columns if col not
    return df
```

```python
# Import necessary PySpark functions and custom module
from pyspark.sql import SparkSession
from data_utils import read_csv_file, transform_dataframe

# Initialize SparkSession (Assuming this is done in Microsoft Fabric Lakehouse environment)
spark = SparkSession.builder.getOrCreate()

# List of datasets with their respective file paths and amount columns
datasets = [
    {"name": "sales", "file_path": "/lakehouse/data/sales_data.csv", "amount_column": "revenue"},
    {"name": "marketing", "file_path": "/lakehouse/data/marketing_data.csv", "amount_column": "cost"},
    {"name": "hr", "file_path": "/lakehouse/data/hr_data.csv", "amount_column": "salary"},
]

# Read and transform each dataset using the imported functions
transformed_dfs = []

for dataset in datasets:
    df = read_csv_file(spark, dataset["file_path"])
    df = transform_dataframe(df, dataset["amount_column"])
    transformed_dfs.append(df)
```

Live demo

Process

- Python project using VS Code
- Package to .whl file
- Publish to Fabric environment
- Import in notebook
- Call and execute methods

Demo time!

What we saw

- Python project & wheel file creation
- Importing into Fabric
- Calling the methods we created

In summary

- Reusable code
  - Write once, use many times
- Less lines of code
  - Reduces technical debt
  - Easier maintenance
- Easier to force development guideliness
  - No ‘handwriting’ for a single developer

Key takeaways

- DRY principle enhances efficiency in software development
- We can take this into data engineering
- Custom Python modules are powerfull tools in Microsoft Fabric
- Implementing an abstract framework makes it easier to develop and maintain your Fabric solution

Bas Land

- BI consultant since 2013
- Co-founder of Kimura Data Intelligence
  - Standardised framework for Fabric
  - Analytical products for several software partners
  - Consultancy (Fabric & Power BI)

- Married to Anouk, we have a daxhund (☺) called Chester, and a baby on the way

- Sports: purple belt Brazilian jiu-jitsu, weight lifting, running

Contact

- Follow me on **LinkedIn**
- Check **ThatFabricGuy.com**
- E-mail **bas.land@kimura.nl**

ThatFabricGuy

LinkedIn

Questions?

Grazie!!!